

DSA in CivicSense Project — File by File Explanation

A direct, comprehensive architectural manual detailing how each Data Structure and Algorithm file is implemented, how the code functions, and how it is applied in the live civic system.

1. `dsa/linked\_list.py` : - Uses Singly Linked List for dynamic complaint storage

Its Code:

```
class Node:
    def __init__(self, data):
        self.data = data          # Stores complaint info (dict: id, title, category)
        self.next = None         # Pointer to the next complaint node

class LinkedList:
    def __init__(self):
        self.head = None        # Starting pointer of the complaint chain
        self._size = 0

    def insert_at_beginning(self, data):
        """Insert urgent complaint at the front (O(1))"""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        self._size += 1

    def insert_at_end(self, data):
        """Add new complaint at the end of the chain (O(n))"""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            curr = self.head
            while curr.next:
                curr = curr.next
            curr.next = new_node
            self._size += 1

    def delete_by_id(self, complaint_id):
        """Remove resolved complaint by its ID (O(n))"""
        if not self.head:
            return None
        if self.head.data.get("id") == complaint_id:
            removed = self.head.data
            self.head = self.head.next
            self._size -= 1
            return removed
        curr = self.head
        while curr.next and curr.next.data.get("id") != complaint_id:
            curr = curr.next
        if curr.next:
            removed = curr.next.data
            curr.next = curr.next.next
            self._size -= 1
            return removed
        return None

    def search_by_id(self, complaint_id):
        """Search for a specific complaint by ID (O(n))"""
        curr = self.head
        while curr:
            if curr.data.get("id") == complaint_id:
                return curr.data
            curr = curr.next
        return None
```

How to Understand This Code:

1. **Node**: Think of each complaint as a train carriage. The carriage holds the complaint details (`self.data`) and a hook (`self.next`) connected to the next complaint carriage.
 2. **self.head**: This is the locomotive / starting point of your complaint chain.
 3. **insert_at_beginning**: Creates a new complaint node and points its `.next` to the current `head`, making it the new first complaint in **O(1)** constant time.
 4. **insert_at_end**: Starts at `head`, travels node-by-node until finding the last node (`curr.next == None`), then attaches the new complaint there.
 5. **delete_by_id**: Traverses the chain until it finds the complaint with matching ID, then bypasses that node (`curr.next = curr.next.next`) to remove it from memory.
-

2. `dsa/queue.py` : - Uses Queue (FIFO) & Priority Queue for complaint dispatching

Its Code:

```
class Queue:
    """Standard First-In-First-Out (FIFO) queue for normal complaints."""
    def __init__(self, max_size=100):
        self.items = []
        self.max_size = max_size

    def enqueue(self, item):
        """Citizen submits complaint -> added to the rear (O(1))"""
        if len(self.items) < self.max_size:
            self.items.append(item)
            return True
        return False

    def dequeue(self):
        """Municipal officer handles complaint -> removed from front (O(n))"""
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def is_empty(self):
        return len(self.items) == 0

class PriorityQueue:
    """Priority Queue for emergency complaints (High > Medium > Low)."""
    PRIORITY_VALUES = {"High": 3, "Medium": 2, "Low": 1}

    def __init__(self, max_size=100):
        self.items = [] # Stores (priority_number, complaint_dict)

    def enqueue(self, item, priority="Medium"):
        """Inserts complaint based on severity level (O(n))"""
        pv = self.PRIORITY_VALUES.get(priority, 2)
        pos = 0
        for i, (p, _) in enumerate(self.items):
            if pv > p:
                pos = i
                break
        pos = i + 1
        self.items.insert(pos, (pv, item))
        return True

    def dequeue(self):
        """Always removes the highest priority emergency complaint first (O(1))"""
        if self.items:
            return self.items.pop(0)[1]
        return None
```

How to Understand This Code:

- FIFO (First-In, First-Out):** Just like people standing in a ticket line, the first citizen who submits a complaint ([enqueue](#)) is the first citizen whose complaint gets resolved ([dequeue](#)).
- Priority Queue:** If an emergency complaint arrives (like [High](#) priority: gas leak or fallen power line), [PriorityQueue.enqueue](#) scans the queue and places it in front of normal ([Medium/Low](#)) complaints so officers handle critical emergencies immediately.

3. `dsa/stack.py` : - Uses Stack (LIFO) for complaint status action history & Undo

Its Code:

```
class Stack:
    """Last-In-First-Out (LIFO) stack for tracking status changes."""
    def __init__(self, max_size=100):
        self.items = []
        self.max_size = max_size

    def push(self, item):
        """Record a new action on top of the stack (O(1))"""
        if len(self.items) < self.max_size:
            self.items.append(item)
            return True
        return False

    def pop(self):
        """Undo the most recent action by removing the top item (O(1))"""
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        """View the latest action without removing it (O(1))"""
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        return len(self.items) == 0
```

How to Understand This Code:

1. **LIFO (Last-In, First-Out)**: Like a stack of plates, the last plate placed on top is the first plate you remove.
2. **Undo Action**: Every time an officer changes a complaint status (e.g., from **Pending** -> **In Progress** -> **Resolved**), the change is **pushed** onto the stack. If an officer made a mistake, clicking **Undo** calls **pop()**, which retrieves the exact last state to revert it.

4. `dsa/infix\_postfix.py` : - Uses Stack Expression Parser (Shunting-Yard) for formula evaluation

Its Code:

```
class InfixPostfix:
    PRECEDENCE = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}

    @classmethod
    def infix_to_postfix(cls, expression):
        """Converts '( A + B ) * C' to 'A B + C *' using an operator stack (O(n))"""
        output = []
        stack = []
        tokens = cls._tokenize(expression)

        for token in tokens:
            if token.isnumeric():
                output.append(token)
            elif token == '(':
                stack.append(token)
            elif token == ')':
                while stack and stack[-1] != '(':
                    output.append(stack.pop())
                if stack:
                    stack.pop() # Pop '('
            elif token in cls.PRECEDENCE:
                while stack and stack[-1] != '(' and cls.PRECEDENCE.get(stack[-1], 0) >= cls.PRECEDENCE[token]:
                    output.append(stack.pop())
                stack.append(token)

        while stack:
            output.append(stack.pop())
        return " ".join(output)

    @classmethod
    def evaluate_postfix(cls, postfix_expr):
        """Evaluates postfix expression using an operand stack (O(n))"""
        stack = []
        for token in postfix_expr.split():
            if token.isnumeric() or token.replace('.', '', 1).isdigit():
                stack.append(float(token))
            elif token in ('+', '-', '*', '/'):
                b = stack.pop()
                a = stack.pop()
                if token == '+': stack.append(a + b)
                elif token == '-': stack.append(a - b)
                elif token == '*': stack.append(a * b)
                elif token == '/': stack.append(a / b if b != 0 else 0)
        return stack[0] if stack else 0
```

How to Understand This Code:

- Why needed:** In the municipal system, urgency scores are computed with dynamic math formulas like (`complaints * 3 + urgency * 2`) / `days`.
- Infix to Postfix:** Computers cannot easily calculate infix expressions with parentheses. This code uses a stack to rearrange numbers and operators in Postfix notation (Reverse Polish Notation) according to math precedence (`*` before `+`).
- Evaluation:** Scans numbers, pushes them onto a stack, and whenever an operator appears, pops two numbers, applies the operation, and pushes the result back.

5. `dsa/iterative.py` : - Uses Iterative Searching & Sorting for complaint tables

Its Code:

```
class IterativeAlgorithms:
    @staticmethod
    def linear_search(arr, target_id, field="id"):
        """Scans complaints one-by-one from start to finish (O(n))"""
        for i, item in enumerate(arr):
            val = item.get(field) if isinstance(item, dict) else item
            if val == target_id:
                return i # Found index
        return -1 # Not found

    @staticmethod
    def binary_search(sorted_arr, target_id, field="id"):
        """Quickly finds complaint in a sorted list by halving the search space (O(log n))"""
        low, high = 0, len(sorted_arr) - 1
        while low <= high:
            mid = (low + high) // 2
            val = sorted_arr[mid].get(field) if isinstance(sorted_arr[mid], dict) else sorted_arr[mid]
            if val == target_id:
                return mid
            elif val < target_id:
                low = mid + 1
            else:
                high = mid - 1
        return -1

    @staticmethod
    def bubble_sort(arr, field="id"):
        """Sorts complaints by repeatedly swapping adjacent out-of-order items (O(n^2))"""
        n = len(arr)
        arr_copy = arr.copy()
        for i in range(n):
            for j in range(0, n - i - 1):
                val1 = arr_copy[j].get(field) if isinstance(arr_copy[j], dict) else arr_copy[j]
                val2 = arr_copy[j + 1].get(field) if isinstance(arr_copy[j + 1], dict) else arr_copy[j + 1]
                if val1 > val2:
                    arr_copy[j], arr_copy[j + 1] = arr_copy[j + 1], arr_copy[j]
        return arr_copy
```

How to Understand This Code:

1. **Linear Search:** Looks at every complaint sequentially. If you have 100 complaints, it may take up to 100 checks.
2. **Binary Search:** Used when complaints are already sorted by ID. It looks at the middle element. If the target is larger, it ignores the entire left half; if smaller, it ignores the right half. It finds any complaint out of 1,000 in only about 10 steps (**O(log n)**).
3. **Bubble Sort:** Compares two adjacent complaints side-by-side. If the left one has a bigger ID than the right one, they swap places.

6. `dsa/recursive.py` : - Uses Recursive Divide-and-Conquer (Merge Sort & Quick Sort)

Its Code:

```
class RecursiveAlgorithms:
    @staticmethod
    def merge_sort(arr):
        """Recursively splits array in halves, sorts, and merges them back (O(n log n))"""
        if len(arr) <= 1:
            return arr

        mid = len(arr) // 2
        left = RecursiveAlgorithms.merge_sort(arr[:mid])    # Recursive call on left half
        right = RecursiveAlgorithms.merge_sort(arr[mid:])    # Recursive call on right half

        # Merge the two sorted halves
        merged = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged
```

How to Understand This Code:

1. **Divide and Conquer:** Instead of sorting 1,000 complaints all at once, Merge Sort divides the list into two lists of 500\$, then four lists of 250\$, all the way down to lists of size 1\$ (Base Case).
 2. **Merge Step:** Then it combines and compares the sorted small lists back together into one fully sorted list in super-fast **O(n log n)** time.
-

7. `app.py` : - Connects all DSA classes to Web APIs

Its Code:

```
# Lines 325-368 in app.py:

@app.route("/api/dsa/queue", methods=["GET", "POST", "DELETE"])
def dsa_queue():
    """API for Queue (FIFO) and Priority Queue operations."""
    if request.method == "POST":
        item = request.json
        live_queue.enqueue(item) # Calls Queue.enqueue()
        return jsonify({"success": True})
    elif request.method == "DELETE":
        item = live_queue.dequeue() # Calls Queue.dequeue()
        return jsonify({"dequeued": item})

@app.route("/api/dsa/stack", methods=["GET", "POST", "DELETE"])
def dsa_stack():
    """API for Stack operations (Push action / Pop undo)."""
    if request.method == "POST":
        live_stack.push(request.json) # Calls Stack.push()
        return jsonify({"success": True})
    elif request.method == "DELETE":
        popped = live_stack.pop() # Calls Stack.pop()
        return jsonify({"popped": popped})
```

How to Understand This Code:

- Whenever the administrator clicks buttons in the web interface (like **Enqueue Complaint**, **Dequeue Complaint**, **Push State**, or **Undo**), the browser makes a `fetch()` HTTP request to these routes in `app.py`, which directly call the corresponding DSA methods on the live data structures.
